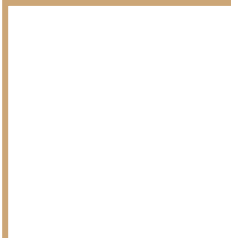


Data Structures

Mushtari Sadia





Linked List



Limitations of Array

- **Fixed capacity** : Once created, an array cannot be resized. The only way to "resize" is to create a larger new array, copy the elements from the original array into the new one, and then change the reference to the new one.

Limitations of Array

- **Fixed capacity** : Once created, an array cannot be resized. The only way to "resize" is to create a larger new array, copy the elements from the original array into the new one, and then change the reference to the new one.
- **Shifting elements in insert** : Since we do not allow gaps in the array, inserting a new element requires that we shift all the subsequent elements right to create a hole where the new element is placed. In the worst case, we "disturb" every slot in the array when we insert it at the beginning of the array!

Limitations of Array

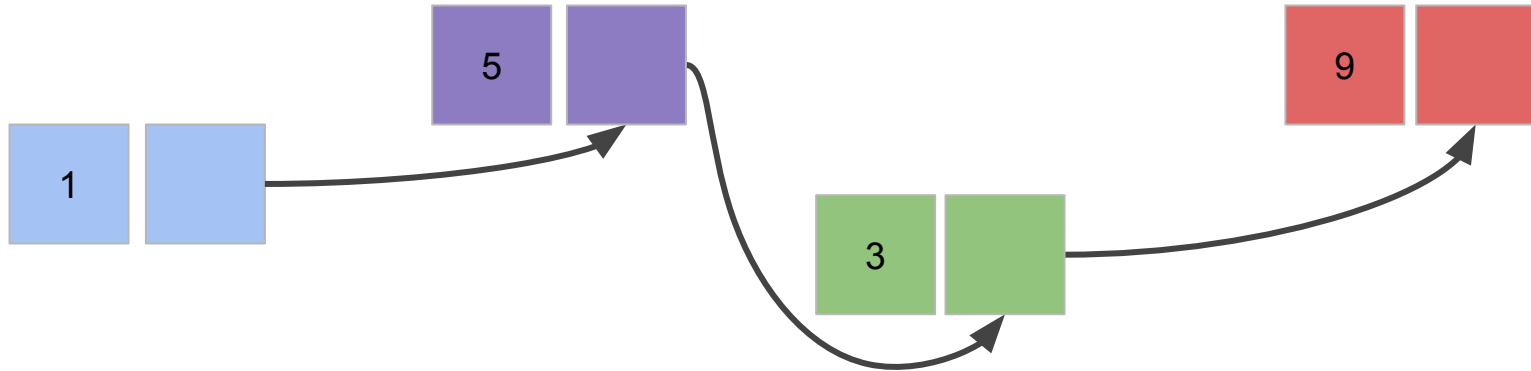
- **Fixed capacity** : Once created, an array cannot be resized. The only way to "resize" is to create a larger new array, copy the elements from the original array into the new one, and then change the reference to the new one.
- **Shifting elements in insert** : Since we do not allow gaps in the array, inserting a new element requires that we shift all the subsequent elements right to create a hole where the new element is placed. In the worst case, we "disturb" every slot in the array when we insert it at the beginning of the array!
- **Shifting elements in removal** : Removing may also require shifting to plug the hole left by the removed element. If the ordering of the elements does not matter, we can avoid shifting by replacing the removed element with the element in the last slot (and then treating the last slot as empty).

Array vs. Linked List

Array
in memory



Linked list
in memory



Resizing - Array

Array
Size 5

arr



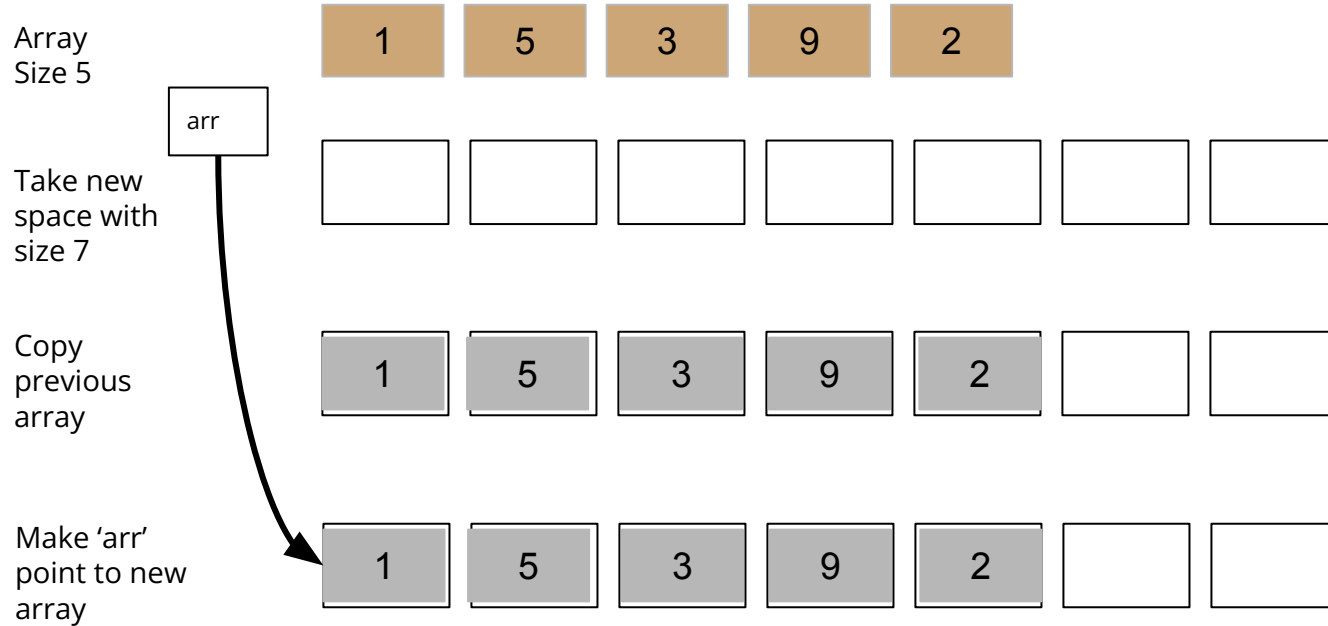
Take new
space with
size 7



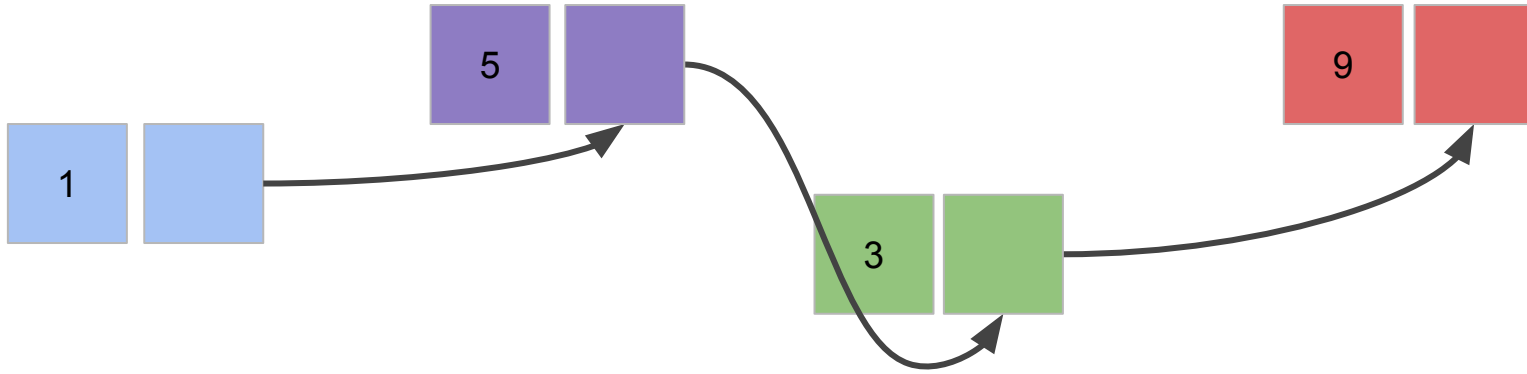
Copy
previous
array



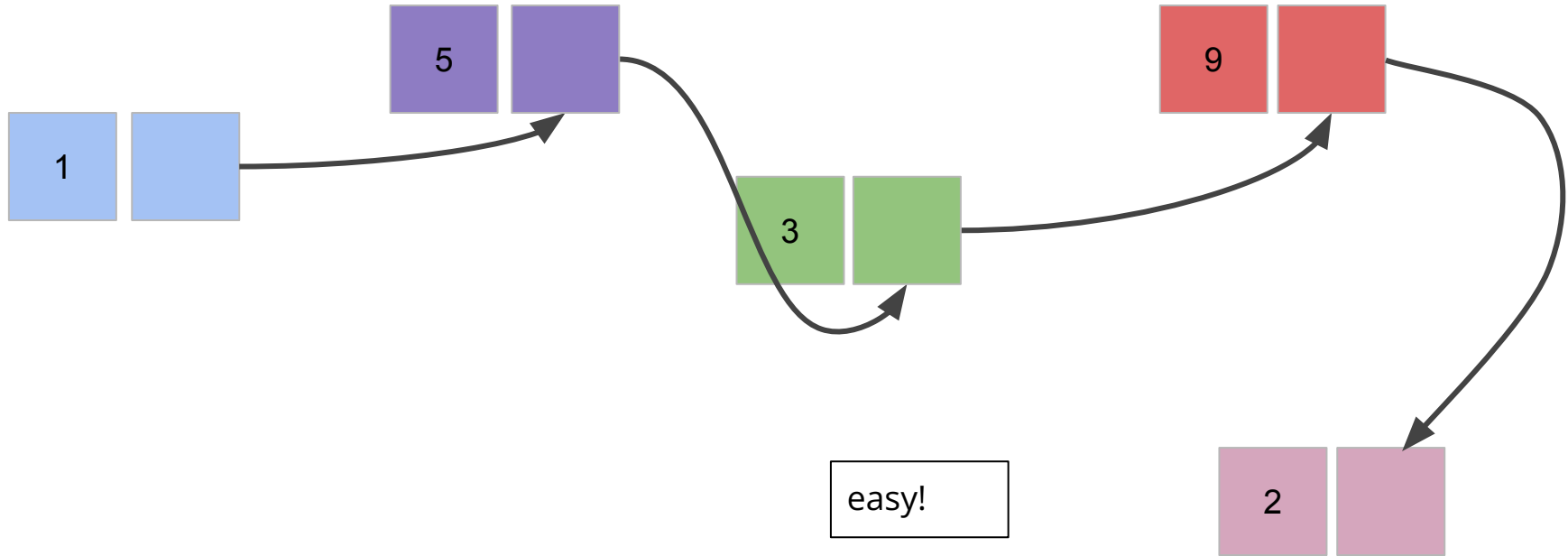
Resizing - Array



Resizing - Linked List



Resizing - Linked List



Insertion - Array

Array
Capacity 6
Size 5



Insert 7 here

Insertion - Array

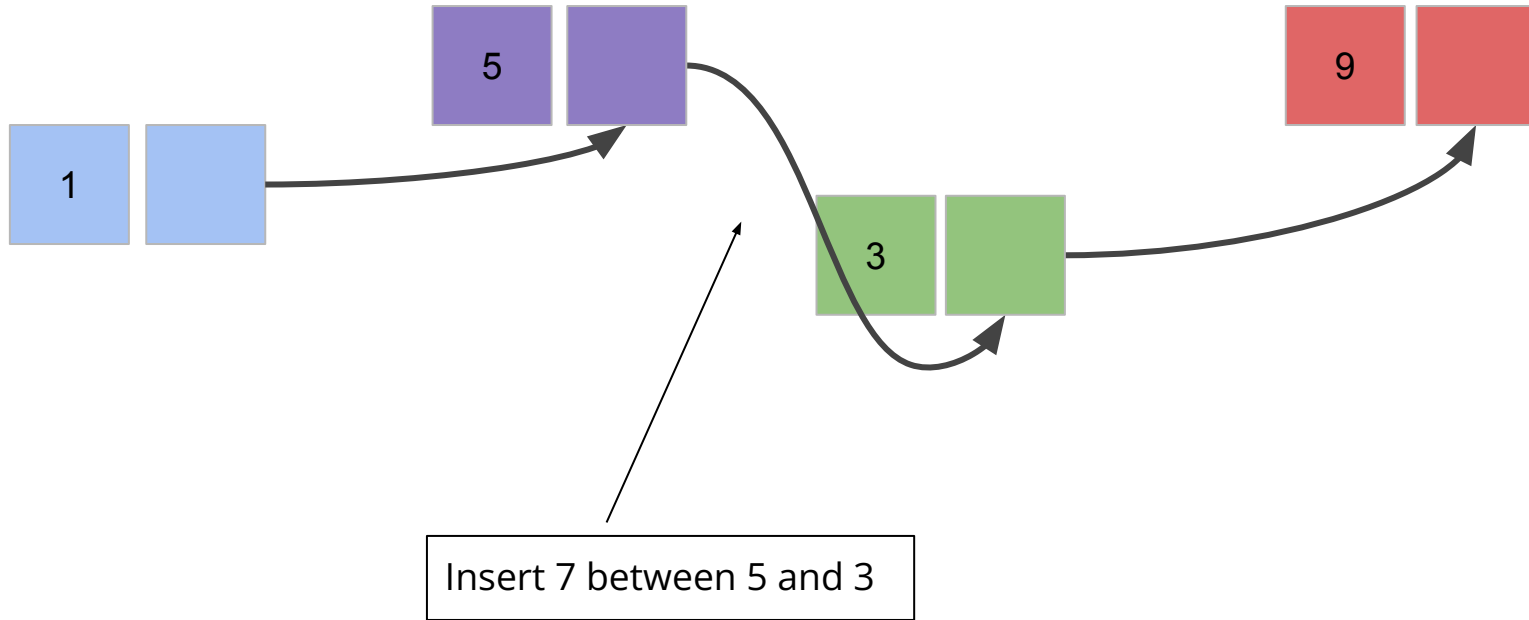
Array
Capacity 6
Size 5



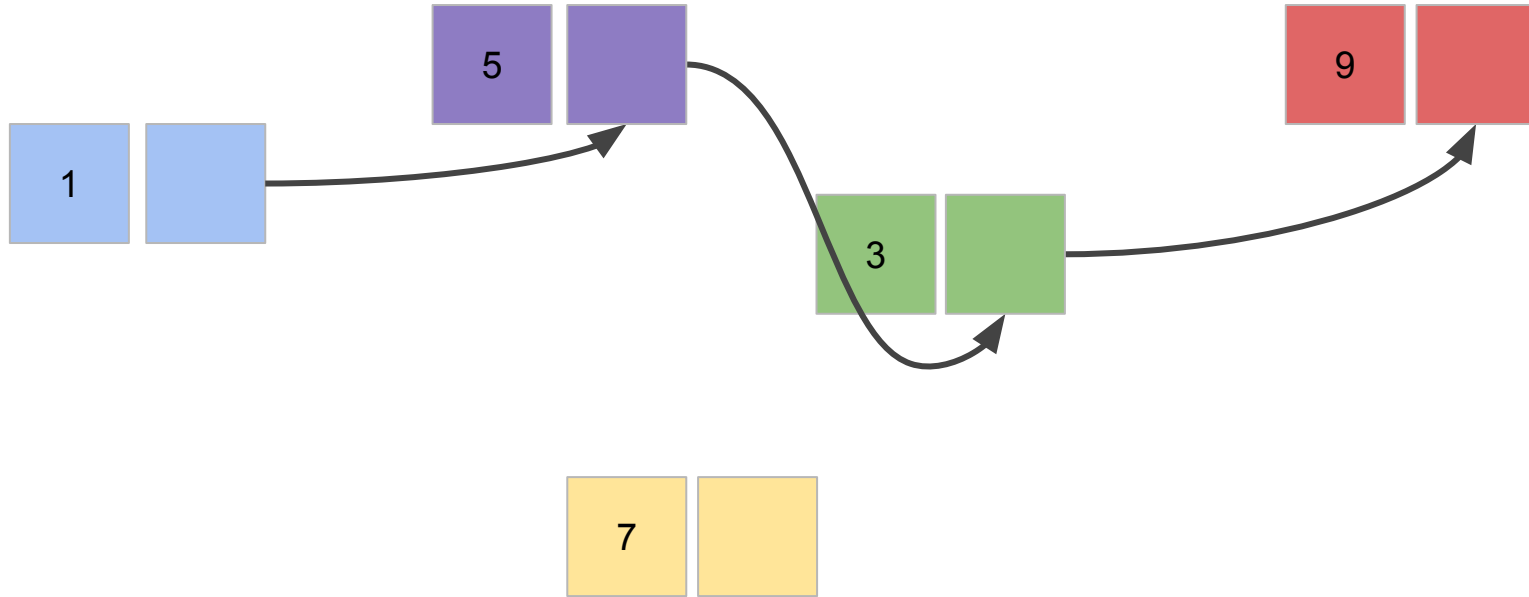
Insert 7 here

`array[3] = 7`

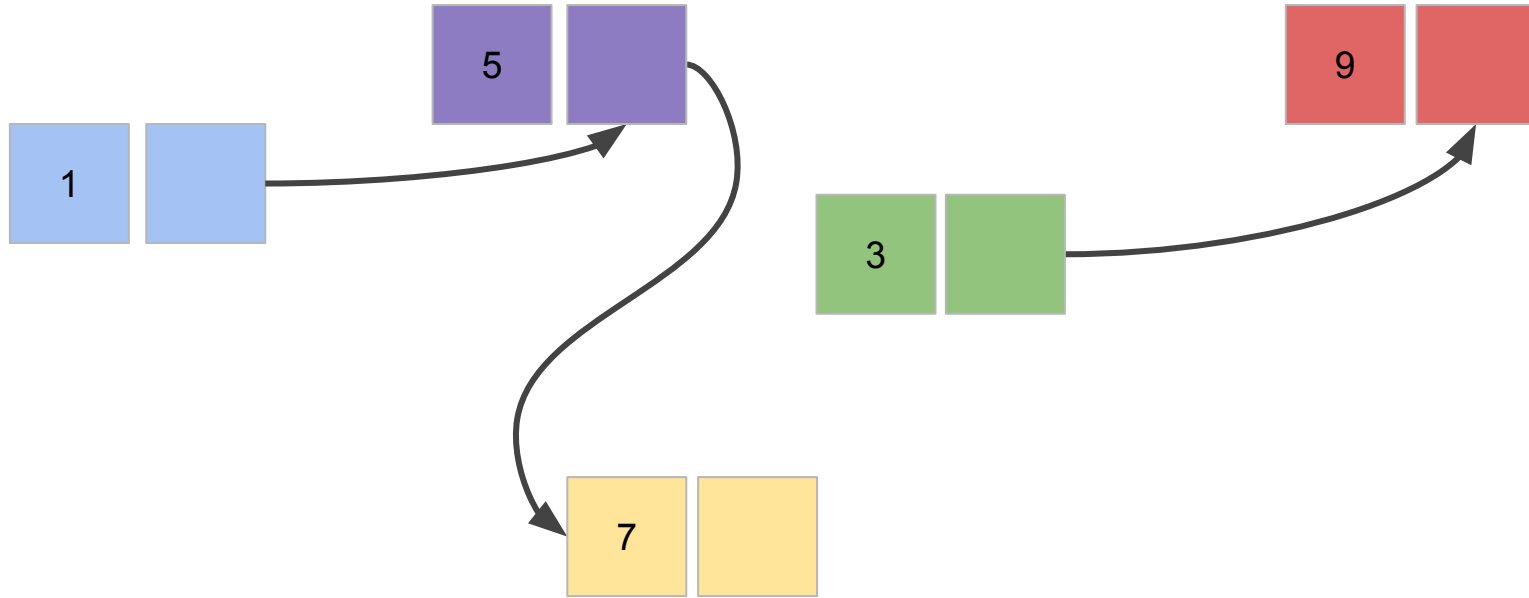
Insertion - Linked List



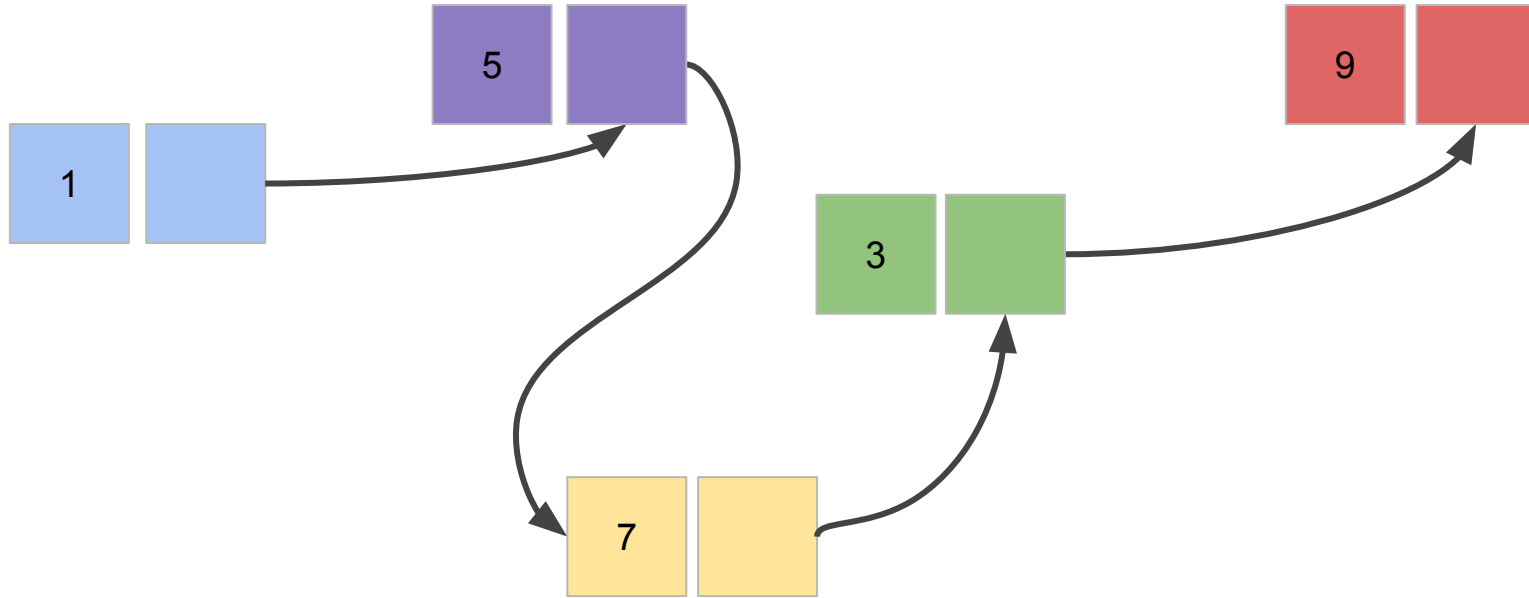
Insertion - Linked List



Insertion - Linked List



Insertion - Linked List



Removal - Array

Array



Delete 5

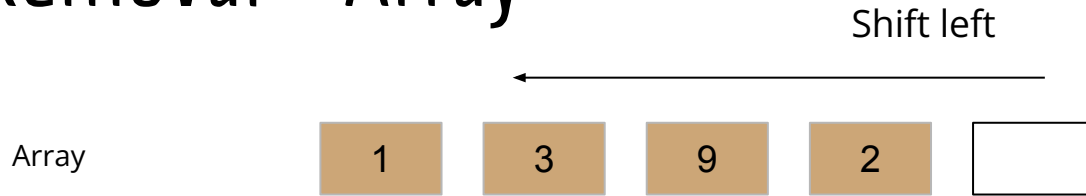
Removal - Array

Array



```
array[1] = None
```

Removal - Array



```
array[1] = None
```

Removal - Array

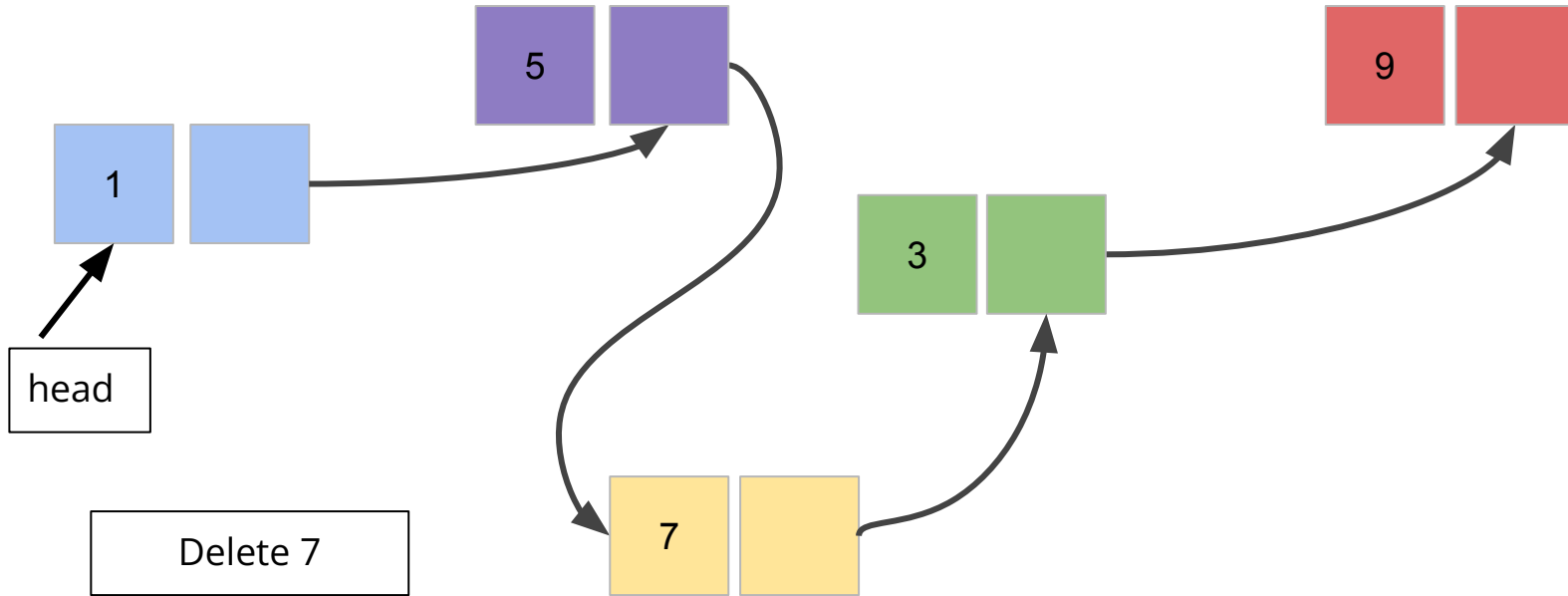
Array
Capacity 6
Size 5



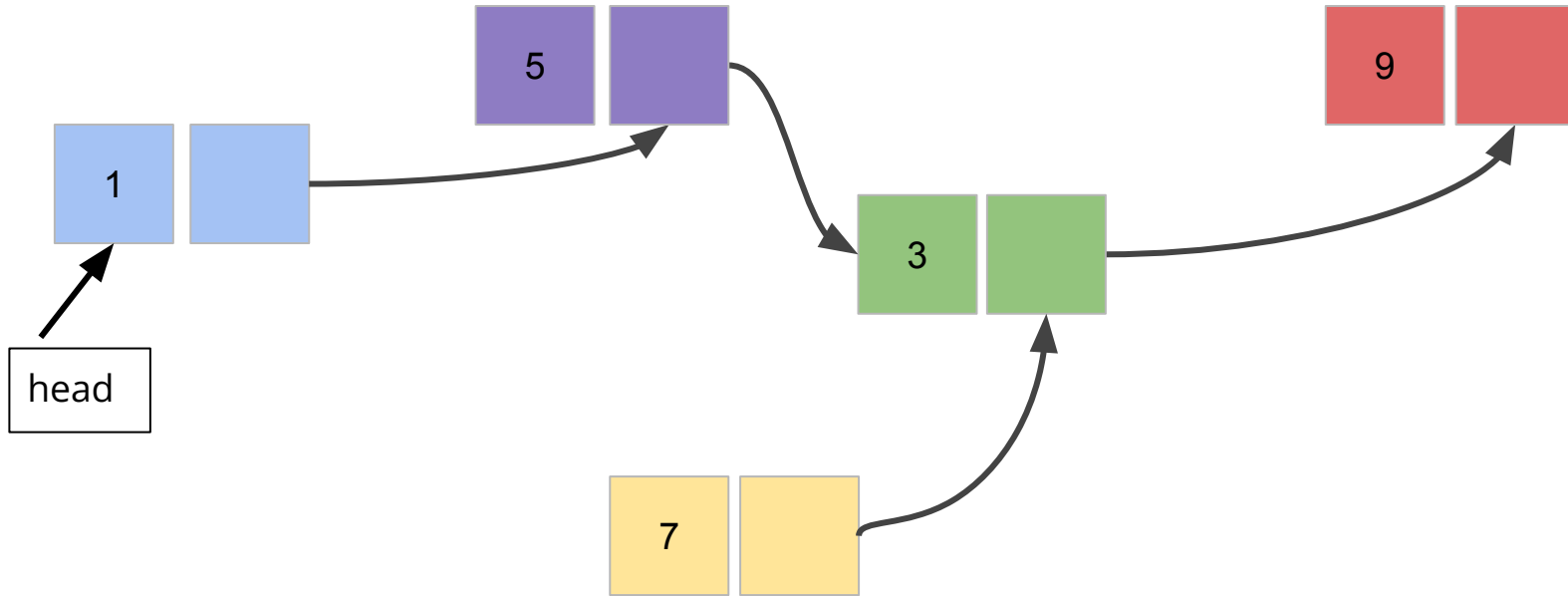
Insert 7 here

`array[3] = 7`

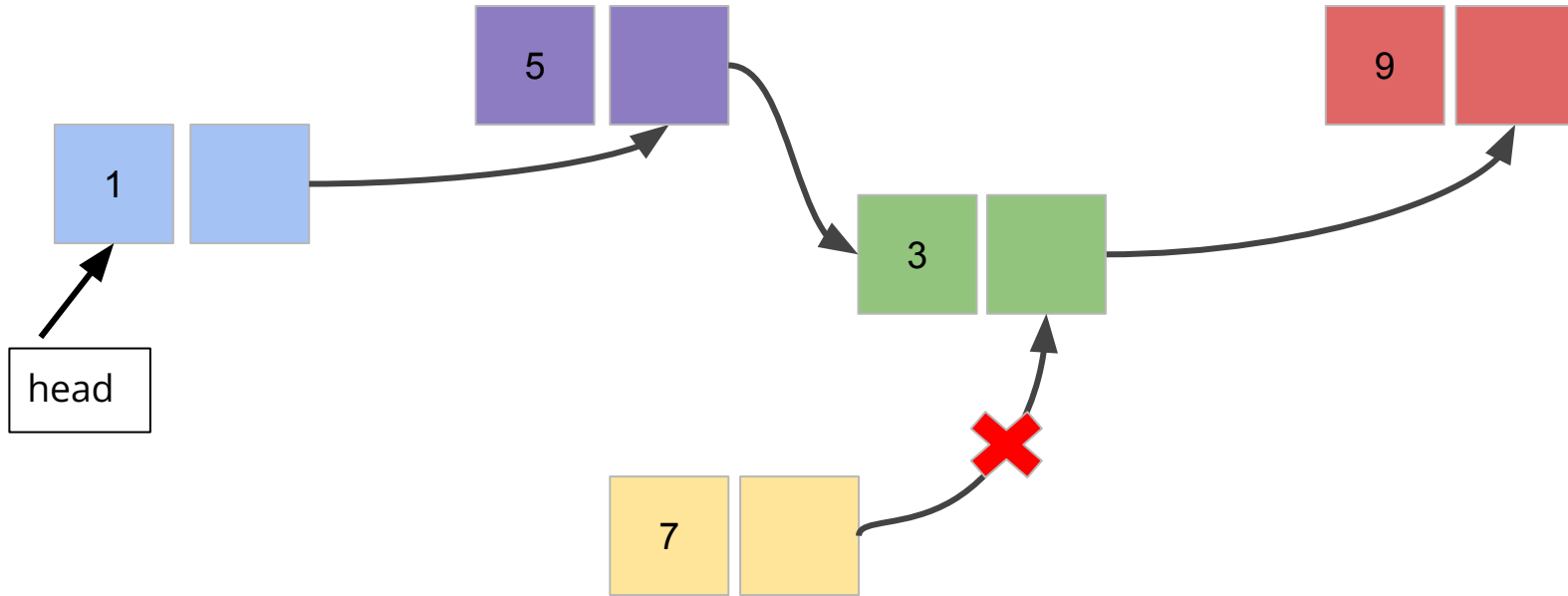
Removal - Linked List



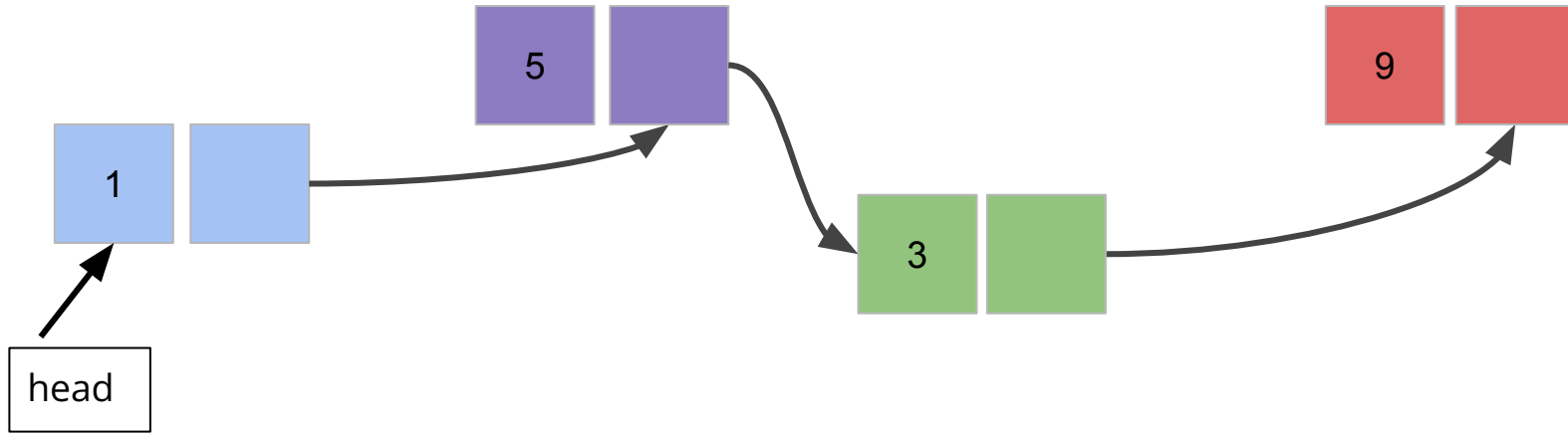
Removal - Linked List



Removal - Linked List

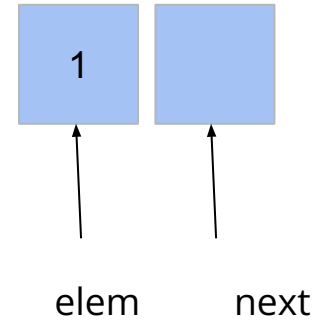


Removal - Linked List



Linked List Creation

```
1 # Node class design
2 class Node:
3     def __init__(self, e, n):
4         self.elem = e
5         self.next = n      (Address of next node)
6
```



Linked List Creation from an Array

```
1 # Creating a list
2 def createList(a):
3     head = Node(a[0], None)
4     tail = head
5     for i in range(1, len(a)):
6         n = Node(a[i], None)
7         tail.next = n
8         tail = tail.next
9     return head
```

Iterating Over a Linked List

```
1 # Iteration over a linked list
2 def iteration(head):
3     temp = head
4     while temp != None:
5         print(temp.element)
6         temp = temp.next
```

Counting elements of a Linked List

```
1 # Counting number of element in the list
2 def count(head):
3     count = 0
4     temp = head
5     while temp != None:
6         count += 1
7         temp = temp.next
8     return count
```

Retrieving an element from an index



```
1 # Getting element of an specific index
2 def elemAt(head, idx):
3     count = 0
4     temp = head
5     obj = None
6     while temp != None:
7         if count == idx:
8             obj = temp.element
9             break
10        temp = temp.next
11        count += 1
12    if obj == None:
13        print("Invalid index")
14    return obj
```

Retrieving a node from an index

```
1 # Getting node of an specific index
2 def nodeAt(head, idx):
3     count = 0
4     temp = head
5     obj = None
6     while temp != None:
7         if count == idx:
8             obj = temp
9             break
10        temp = temp.next
11        count += 1
12    if obj == None:
13        print("Invalid index")
14    return obj
```

Update value of specific index

```
1 # Setting new element of an specific index
2 def set(head, idx, elem):
3     count = 0
4     temp = head
5     isUpdated = False
6     while temp != None:
7         if count == idx:
8             temp.elem = elem
9             isUpdated = True
10            break
11        temp = temp.next
12        count += 1
13    if isUpdated:
14        print("Value successfully updated!!!!")
15    else:
16        print("Invalid index")
```

Searching an element in the list

```
1 def contains(head, elem):  
2     temp = head  
3     while temp != None:  
4         if elem == temp.elem:  
5             return True  
6         temp = temp.next  
7     return False
```


Searching an element in the list

```
1 # Getting index of an specific element
2 def indexOf(head, elem):
3     temp = head
4     count = 0
5     while temp != None:
6         if elem == temp.elem:
7             return count
8         count += 1
9         temp = temp.next
10    return -1 # Here -1 represents the absence of element in the list
```

Inserting an element into a list

```
1 def insert(head, elem, idx):
2     total_nodes = count(head)
3     if idx == 0: # Inserting at the beginning
4         n = Node(elem, head)
5         head = n
6     elif idx >= 1 and idx < total_nodes: # Inserting at the middle
7         n = Node(elem, head)
8         n1 = nodeAt(head, idx - 1)
9         n2 = nodeAt(head, idx)
10        n.next = n2
11        n1.next = n
12    elif idx == total_nodes: # Inserting at the end
13        n = Node(elem, None)
14        n1 = nodeAt(head, total_nodes - 1)
15        n1.next = n
16    else:
17        print("Invalid Index")
18    return head
```

Removing an element from a list

```
1 def remove(head, idx):
2     if idx == 0: # Removing first element
3         head = head.next
4     elif idx >= 1 and idx < count(head): # Removing middle element
5         n1 = nodeAt(head, idx - 1)
6         removed_node = n1.next
7         n1.next = removed_node.next
8     else:
9         print("Invalid Index")
10    return head
```

Copying a list

```
1 def copyList(source):
2     copy_head = None
3     copy_tail = None
4     temp = source
5     while temp != None:
6         n = Node(temp.elem, None)
7         if copy_head == None:
8             copy_head = n
9             copy_tail = copy_head
10        else:
11            copy_tail.next = n
12            copy_tail = copy_tail.next
13        temp = temp.next
14    return copy_head
```

Reversing a list - out of place

```
1 def reverse_out_of_place(head):  
2     new_head = Node(head.elem, None)  
3     temp = head.next  
4     while temp != None:  
5         n = Node(temp.elem, new_head)  
6         new_head = n  
7         temp = temp.next  
8     return new_head
```

Reversing a list - in place

```
1 def reverse_in_place(head):  
2     new_head = None  
3     temp = head  
4     while temp != None:  
5         n = temp.next  
6         temp.next = new_head  
7         new_head = temp  
8         temp = n  
9     return new_head
```

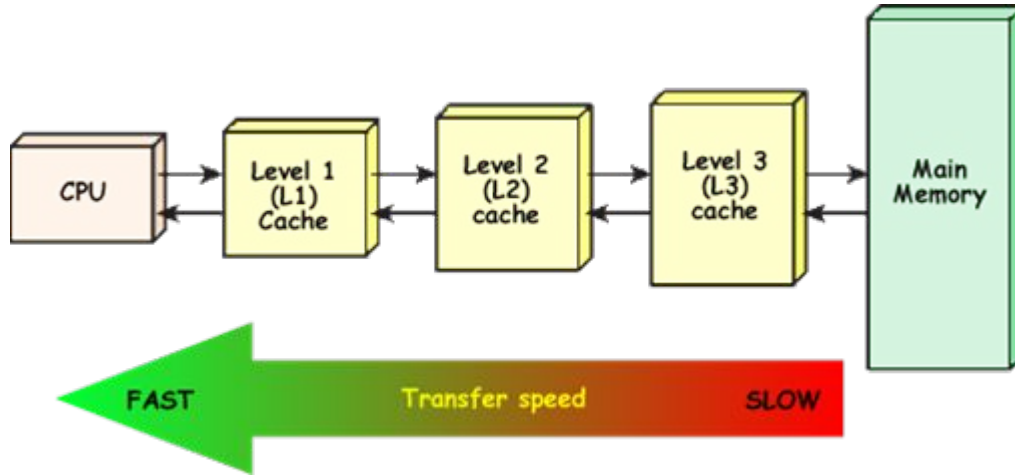

Rotating a list - left

```
1 def rotate_left(head):  
2     new_head = head.next  
3     temp = new_head  
4     while temp.next != None:  
5         temp = temp.next  
6     temp.next = head  
7     head.next = None  
8     head = new_head  
9     return head
```

Rotating a list - right

```
1 def rotate_right(head):  
2     last_node = head.next  
3     second_last_node = head  
4     while last_node.next != None:  
5         last_node = last_node.next  
6         second_last_node = second_last_node.next  
7     last_node.next = head  
8     second_last_node.next = None  
9     head = last_node  
10    return head
```


Array Vs Linked Lists



Array Vs Linked Lists

From the early days of computers, CPU is many times faster than the RAM (which is the memory) which in turn many times faster than the DISK (which is the computer's data storage). Since, to do any computation in the CPU, you need data; the computer designers applied a simple optimization to avoid hurting computer's performance due to the slowness of data read/write. The idea is that the RAM loads a large block of data at once, which is several KB or MB, from the storage even if the CPU asks for a single byte of data. The reason is, we expect in any typical program, CPU will ask for more data from locations nearby to its first request. Surprisingly, or not, this assumption happens to be true for more than 98% of time for all data accesses of a program. However, even with this strategy, hardware designer realized that the slowness of the RAM (no longer the slowness of the disk, which has been taken care of) hurts overall program performance greatly. So they just reused the same idea by introducing something called caches. Caches are like RAM but much smaller and faster (their small size is related to their faster speed).

Array Vs Linked Lists

When a program runs, if the CPU ask for a single byte of data that is available in the RAM, the hardware actually load multiple consecutive bytes, at the range of 256 bytes or more from the RAM to the cache. The reason is the same; the hardware hopes that the program will make request for data among the additional bytes in near future. At that time, the request can be served from the cache. This idea of loading more data in closer memory works so good that now a days, all computers has multiple levels of caches between the CPU and the RAM.

Array Vs Linked Lists

This strategy for optimization affects the performance of element traversals of an array and a list differently. As you know, array elements are allocated in consecutive memory cells during the creation of the array. Therefore, traversal is extremely fast. There are just occasional data transfers between the cache and the RAM. However, space for a list element is allocated when you add the element to the list. The hardware gives whatever the next free memory cells available at that time. As a result, the elements are actually in different places of the RAM and are only connected together by the links of the linked list. So when you traverse a linked list, the hardware has to fetch data from the RAM to caches many times more. Therefore, traversing a linked list is significantly slower than traversing a linear array of the same size.

Different Types of Linked Lists

Category 1	Category 2	Category 3
Non-Dummy Headed	Singly	Linear
Dummy Headed	Doubly	Circular

Non Dummy Headed

$10 \rightarrow 20 \rightarrow 30 \rightarrow 40$

First node contains value 10.

Dummy Headed

DH $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ 40

In this linked list, DH refers dummy head which is a node without any element. The first item is stored after the dummy head.

Singly

$10 \rightarrow 20 \rightarrow 30 \rightarrow 40$

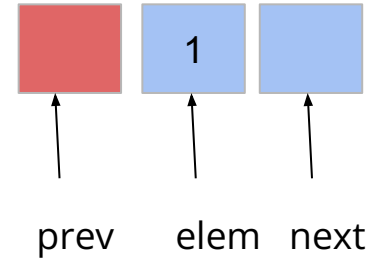
Only contains next pointer, no
previous pointer

Doubly

10 ⇌ 20 ⇌ 30 ⇌ 40

Contains both next and previous pointer

```
class Node:  
    def __init__(self, e, n, p):  
        self.elem = e  
        self.next = n  
        self.prev = p
```



Linear vs Circular

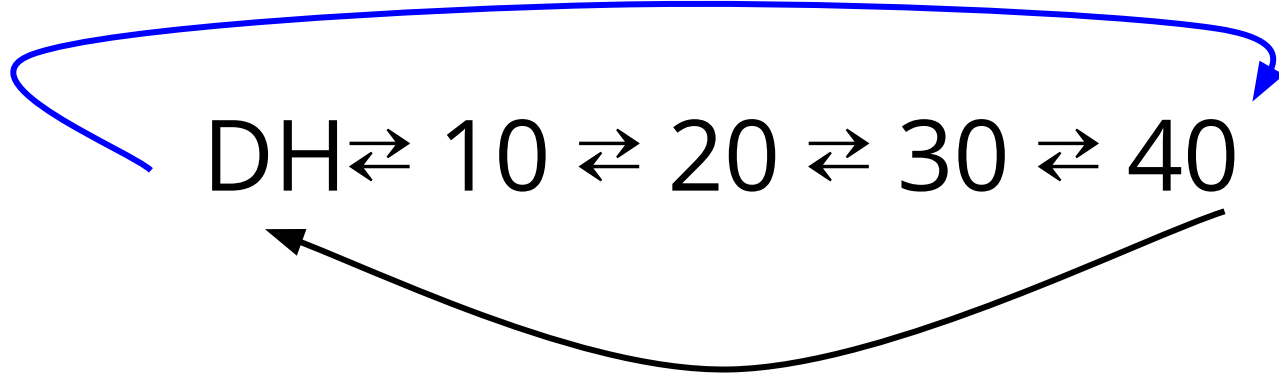
Linear: $10 \rightarrow 20 \rightarrow 30 \rightarrow 40$

Circular:

$10 \rightarrow 20 \rightarrow 30 \rightarrow 40$



Dummy-headed doubly circular linked list



Dummy-headed doubly circular linked list

```
class DoublyNode:
```

```
    def __init__(self, elem, next, prev):
```

```
        self.elem = elem
```

```
        self.next = next # To store the next node's reference.
```

```
        self.prev = prev # To store the previous node's reference.
```

Dummy-headed doubly circular linked list

```
1 def createList(a):
2     dh = DoublyNode(None, None, None)
3     dh.next = dh
4     dh.prev = dh
5     tail = dh
6
7     for i in range(len(a)):
8         n = DoublyNode(a[i], dh, tail)
9         tail.next = n
10        tail = tail.next
11        dh.prev = tail
12
13    return dh
```

Dummy-headed doubly circular linked list

```
1 def iteration(dh):  
2     temp = dh.next  
3     while temp != dh:  
4         print(temp.elem)  
5         temp = temp.next
```

Dummy-headed doubly circular linked list

```
1 def nodeAt(dh, idx):
2     temp = dh.next
3     c = 0
4     while temp != dh:
5         if c == idx:
6             return temp
7         c += 1
8         temp = temp.next
9     return None # Invalid Index
```

Dummy-headed doubly circular linked list

```
1 def insertion(dh, elem, idx):
2     # Assuming the idx is valid
3     node_to_insert = DoublyNode(elem, None, None)
4     indexed_node = nodeAt(dh, idx) # Retriving the node at that index
5     prev_node = indexed_node.prev # There will always be a previous node
6     # Change the connection
7     # Observe that no special case is needed
8     node_to_insert.next = indexed_node
9     node_to_insert.prev = prev_node
10    prev_node.next = node_to_insert
11    indexed_node.prev = node_to_insert
```


Dummy-headed doubly circular linked list

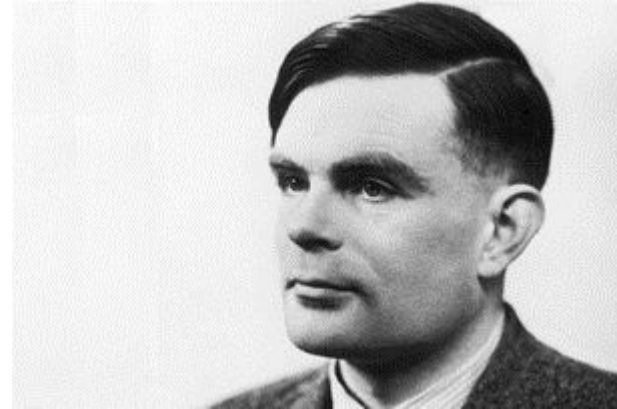
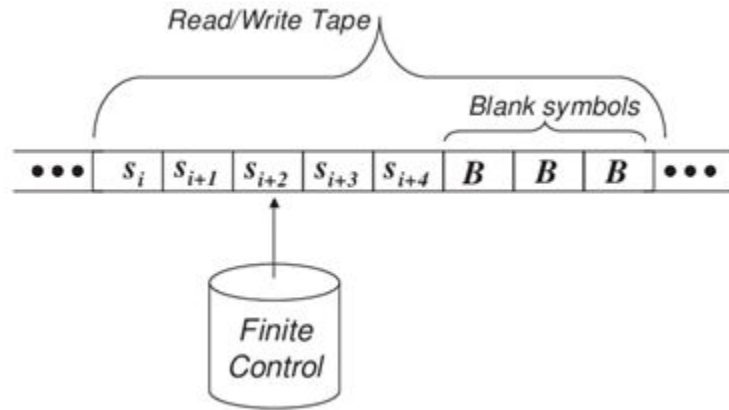
```
1 def removal(dh, idx):
2     # Assuming the idx is valid
3     node_to_remove = nodeAt(dh, idx)
4     prev_node = node_to_remove.prev
5     next_node = node_to_remove.next
6     # Change the connection
7     # No special case is needed
8     prev_node.next = next_node
9     next_node.prev = prev_node
10    node_to_remove.next = None
11    node_to_remove.prev = None
12    return node_to_remove.elem # Returning the removed element
```

Reasons for doubly linked list

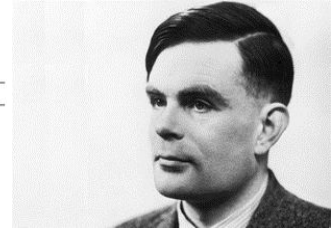
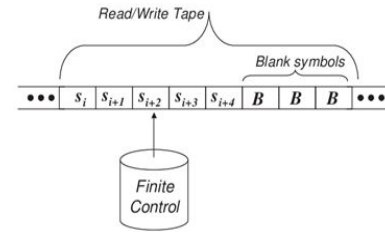
Problem of singly linked lists:

- We have to be very careful regarding where to stop the traversal. For example, if you want to insert at position n , then we have to stop traversal at position $n - 1$. If you want to read then we stop traversal at n . If we make one more wrong step then we cannot backtrack. You have to restart from the head.

Reasons for doubly linked list



Reasons for doubly linked list



Some of you know about the famous mathematician and computer scientist Alan Turing. He investigated and proved many important properties about computers. One of his most fascinating contributions is the Turing Machine that can simulate any computer. By doing so, the Turing machine allows us to investigate the fundamental limitations and capabilities of all computers ever being built. The machine is surprisingly simple. It has an infinite tape of symbols and a read-write head that can make one-step forward or backward on that tape in each step or change the content of the tape cell where the head is located at that moment. The read-write head, also called the finite control, keeps track of its current state and decides what to do based-on the symbol on the tape in its position and its state.

So what kind of list you need to implement this tape?